

Container mit Schwerpunkt Kubernetes

Weitere Objekte



- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Annotations
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

- Labels ...
 - werden verwendet, um Ressourcen zu organisieren, kategorisieren und zu kennzeichnen.
 - sind **Key-Value-Paare**.
 - können beliebigen Kubernetes-Objekten (z. B. Pods) zugewiesen werden.
 - werden in den **Metadaten** spezifiziert.
 - dienen der **Identifizierung von Objekten**, z. B. allen Produktions-Objekten das Label `env: prod` zuweisen.
 - Erlauben eine dynamische Auswahl zugehöriger Objekte (Service → zu nutzende Pods).

- Beispiel:

```
apiVersion: v1
kind: Pod
metadata:
  name: test-pod
  labels:
    env: prod
...
```

Label
Selektor

```
apiVersion: v1
kind: Service
metadata:
  name: test-service
spec:
  selector:
    env: prod
  ports:
  ...
```

- Labels der Pods ausgeben:

```
# kubectl get po --show-labels
```

NAME	READY	STATUS	RESTARTS	AGE	LABELS
myapp-pod	1/1	Running	0	13m	app=myapp

- Label als Spalte ausgeben:

```
# kubectl get po -L app
```

NAME	READY	STATUS	RESTARTS	AGE	APP
myapp-pod	1/1	Running	0	12m	myapp

- Pods mit einem bestimmten Label ausgeben:

```
# kubectl get po -l app=myapp
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-pod	1/1	Running	0	12m

- Nachträgliches Anpassen eines Labels einer Ressource:

```
# kubectl label <ressource> <name> <key_n>=<value_n>
```

Empfohlene Labels für Ressourcen

Key	Beschreibung
app.kubernetes.io/name	Name der Applikation
app.kubernetes.io/instance	Ein eindeutiger Name, mit dem die Instanz der Applikation identifiziert werden kann
app.kubernetes.io/version	Die aktuelle Version der Applikation
app.kubernetes.io/component	Welche Komponente innerhalb der Architektur ist die Applikation
app.kubernetes.io/part-of	Name der übergeordneten Applikation
app.kubernetes.io/managed-by	Das Tool, das zur Verwaltung des Betriebs der Applikation genutzt wird

Beispiel:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app.kubernetes.io/name: "my-app"
    app.kubernetes.io/instance: "my-app-instance"
    app.kubernetes.io/version: "v1.0.0"
    app.kubernetes.io/component: "backend"
    app.kubernetes.io/part-of: "my-app-suite"
    app.kubernetes.io/managed-by: "helm"
spec:
  replicas: 3
  selector:
    matchLabels:
      app.kubernetes.io/name: "my-app"
  template:
    metadata:
      labels:
        app.kubernetes.io/name: "my-app"
        app.kubernetes.io/instance: "my-app-instance"
        app.kubernetes.io/version: "v1.0.0"
        app.kubernetes.io/component: "backend"
        app.kubernetes.io/part-of: "my-app-suite"
        app.kubernetes.io/managed-by: "helm"
    ...
```

Annotations (1/2)

- Annotations sind **Key/Value-Paare**.
- Möglichkeit, **nicht identifizierende Metadaten** für Objekte zu hinterlegen.
- Können von Tools und Libraries genutzt werden.
- Werden z. B. für die Konfiguration von Services (LoadBalancer) und Ingresses verwendet.
- Können auch zum Ablegen ...
 - von Skalierungsinformationen,
 - des Grundes für ein Update,
 - von Build, Release oder Image Informationen,
 - der E-Mail-Adresse der verantwortlichen Person... genutzt werden.
- Beispiel:

```
metadata:  
  annotations:  
    email: "info@ordix.de"
```

- **Vergleich mit Labels:**

Labels dienen der Identifikation und Auswahl von Ressourcen. Annotations der Bereitstellung weiterer Informationen, die von Kubernetes selbst nicht interpretiert werden müssen.

- **Verwendung von Annotations:**

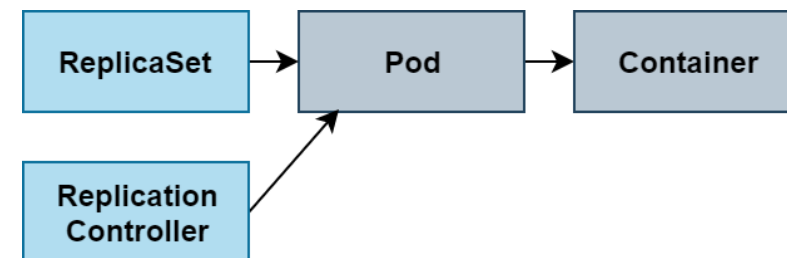
- **Automatisierung:** Tools können Annotations verwenden, um spezifische Informationen zu Ressourcen zu erfassen, z.B. Deployment-Prozesse.
- **Überwachung:** Monitoring-Systeme wie Prometheus können Annotations verwenden, um Metriken oder Logs zu bereichern.
- **Dokumentation:** Entwickler können Annotations nutzen, um zusätzliche Kontextinformationen zu Ressourcen hinzuzufügen.

- **Annotations:** Detaillierte Informationen. Das können auch längere Texte oder JSON-Strukturen sein.

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  annotations:
    description: "This is my pod"
    owner: "team-a"
spec:
  containers:
  - name: mycontainer
    image: nginx
```

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

- **Pods** werden in der Regel **in Verbindung mit** einem *ReplicationController* / **ReplicaSet** eingesetzt, Gründe:
 - **Redundanz**
 - **Skalierung**
- *ReplicaSets* sind die neuere Variante der *ReplicationController*.
- Ein ReplicaSet-Controller kann als clusterweiter Pod-Manager verstanden werden.
- Reconciliation Loop: Überwacht, dass die angegebene Anzahl von Repliken eines Pods existiert.
 - Starten bzw. Beenden von Pods, wenn Anzahl nicht stimmt.
- *ReplicaSet* entspricht einem **einzigen, skalierbaren Microservice**.



Replica-Set – Manifest

- Jedes *ReplicaSet* hat einen **eindeutigen Namen** (metadata.name).
- *spec.replicas*: Definiert die Anzahl der Pods.
- *spec.selector*: Selektiert die Pods anhand der angegebenen Label.
- *spec.template*: Spezifiziert, wie der Pod des Replica-Sets aufgebaut ist.
- **Hinweis**: Haben andere Pods zufälligerweise dasselbe Label, werden auch sie zum Replica-Set gezählt.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
```

kubectl – Arbeiten mit ReplicaSets

- Auflisten der ReplicaSets:

```
# kubectl get replicaset | rs
```

- Details eines ReplicaSets anzeigen:

```
# kubectl describe rs <name>
```

- Anzahl der Replicas anpassen:

```
# kubectl scale rs/test-rs --replicas=2
```

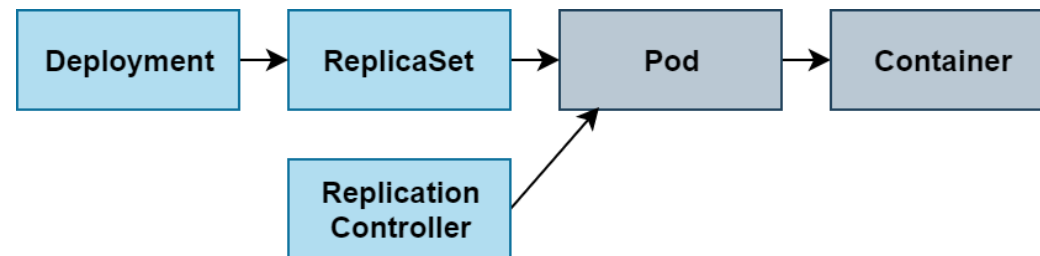
- Löschen von ReplicaSets, der GarbageCollector löscht die zugehörigen Pods automatisch:

```
# kubectl delete rs test-rs
```

```
# kubectl delete rs <name> [--cascade=orphan/false]
```

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- **Anwendungs-Deployment (Deployment)**
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

- Pods und ReplicaSets sind an ein spezifisches Image gebunden.
- ReplicaSets werden meist **in *Deployments* genutzt**.
- Der Deployment-Controller ermöglicht Updates von Pods und ReplicaSets mittels angegebener **Deployment-Strategie**.
 - RollingUpdate
 - Recreate



Deployments – Manifest

- Eigenschaften in Zusammenhang mit einem Deployment.
- *spec.replicas* (default: 1): Größe des Clusters, Anzahl der Replicas.
- *strategy.type* (default: Rolling): Unterbrechungsfreier Betrieb durch Parallelbetrieb von alter und neuer Version oder Abschalten und Neustarten (Recreate).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.18
          ports:
            - containerPort: 80
```

Deployments → ReplicaSets → Pods

```
# kubectl get deployments | deploy
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
nginx-deploy	3	3	3	3	1m

```
# kubectl get rs
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-deploy-f9bfc9574	3	3	3	1m

```
# kubectl get po -l app=nginx
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deploy-f9bfc9574-9krdz	1/1	Running	0	1m
nginx-deploy-f9bfc9574-mx5jx	1/1	Running	0	1m
nginx-deploy-f9bfc9574-nnqvz	1/1	Running	0	1m

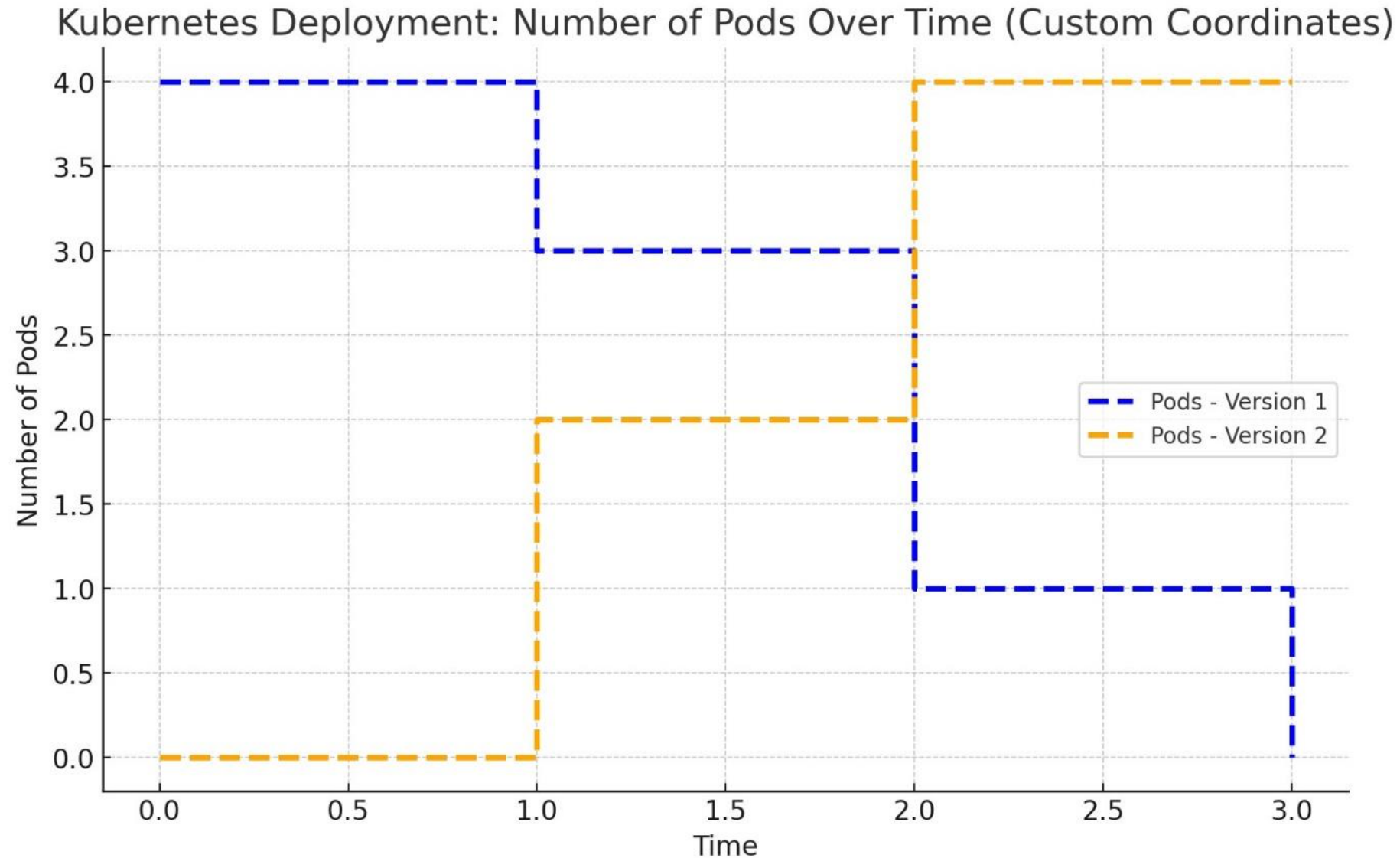
Recreate

- Die **einfachste Rollout-Strategie**. Ist mit einer Downtime verbunden.
- ReplicaSet wird angepasst, um das neue Image zu verwenden.
- **Pods werden terminiert und** durch das ReplicaSet automatisch **mit dem neuen Image gestartet**.

Rolling Update (Default)

- Die zu präferierende Strategie, da **keine Downtime auftritt**.
- **Pods werden inkrementell nacheinander aktualisiert**.
- Verschiedene Versionen sind gleichzeitig aktiv. Muss von der Software unterstützt werden!
- Konfiguration via: *spec.strategy.rollingUpdate*
 - *maxSurge* max. Pods über der gewünschten Anzahl (Default: 25%)
 - *maxUnavailable* max. Pods unerreichbar während des Updates (Default: 25%)

Rolling Deployment mit 4 Pods - Default-Einstellungen



Updaten von Deployments

- Deployment ausrollen nach Manifest-Änderung:
kubectl apply -f <Manifest>
- Alternativ: Image des Containers updaten:
*# kubectl set image deploy nginx-deploy
nginx=nginx:1.19*
- Das letzte Deployment 'beschriften':
*# kubectl annotate deployment nginx-deploy
kubernetes.io/change-cause="Initial version"*

*# kubectl rollout history deployment nginx-deploy
deployment.apps/nginx-deploy
REVISION CHANGE-CAUSE
1 Initial version*

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  labels:
    app: nginx
  annotations:
    kubernetes.io/change-cause: "v1.19"
spec:
  ...
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.19
        ports:
        - containerPort: 80
```

- Beobachten des Rollout-Status:

```
# kubectl describe deploy nginx-deploy
```

```
# kubectl rollout status deploy nginx-deploy
```

```
Waiting for deployment "nginx-deploy" rollout to finish: 0 of 3 updated replicas are available...
```

```
Waiting for deployment "nginx-deploy" rollout to finish: 1 of 3 updated replicas are available...
```

- Auflisten der Rollout-History bzw. Anzeigen einer spezifizierten Revision:

```
# kubectl rollout history deploy nginx-deploy
```

```
# kubectl rollout history <type> <name> [--revision=<nr>]
```

- Rollback zu einer früheren Version:

```
# kubectl rollout undo deploy nginx-deploy  
# kubectl rollout undo <type> <name> [--to-revision=<nr>]
```

- Rollout pausieren und fortsetzen:

In bestimmten Situationen möchten Sie möglicherweise einen Rollout pausieren, um weitere Überprüfungen vorzunehmen oder manuelle Eingriffe zu ermöglichen. Danach können Sie den Rollout fortsetzen.

```
# kubectl rollout pause deployment/nginx-deploy  
# kubectl rollout resume deployment/nginx-deploy
```

- Rollout neu starten:

Wird verwendet, um ein Deployment, StatefulSet oder DaemonSet neu zu starten. Dies ist nützlich, um eine Neuverteilung der Pods zu erzwingen, ohne die Spezifikationen der Ressource zu ändern. Das ist besonders hilfreich, wenn eine neue Version eines Images bereitgestellt oder eine Konfigurationsänderung vorgenommen wurde, die Pods neu starten soll, ohne dass eine vollständige Aktualisierung oder Neudefinition der Ressource erforderlich ist.

```
# kubectl rollout restart deployment/nginx-deploy
```

Beim Rollback eines Deployments in Kubernetes wird die Anzahl der **Replicas** nicht zurückgesetzt, weil Kubernetes das **Rollout-Konzept** so gestaltet hat, dass nur die Spezifikationen des **Pod Templates** zurückgesetzt werden.

Das bedeutet:

1. Fokus auf Pod-Spezifikationen:

- Ein Rollback bezieht sich in erster Linie auf die Änderungen, die am Pod Template vorgenommen wurden, wie z.B. Änderungen am Container-Image, an den Umgebungsvariablen oder an den Volumes.
- Die Anzahl der Replicas ist eine Eigenschaft des Deployments, die üblicherweise unabhängig von den Pod-Spezifikationen verwaltet wird. Sie wird oft dynamisch angepasst, z.B. durch Horizontal Pod Autoscalers (HPA) oder manuell durch Administratoren.

2. Beibehaltung des Skalierungszustands:

- Kubernetes behält die aktuelle Anzahl der Replikate bei, um den zuletzt gewünschten Skalierungszustand nicht zu überschreiben. Wenn die Anzahl der Replikate zurückgesetzt würde, könnte dies unbeabsichtigte Auswirkungen auf die Verfügbarkeit der Anwendung haben.
- Beispiel: Wenn ein Deployment während eines Rollouts von 3 auf 5 Replicas skaliert wurde und dann ein Rollback erfolgt, möchte man normalerweise die 5 Replicas beibehalten, um sicherzustellen, dass die Anwendung weiterhin die erwartete Last bewältigen kann.

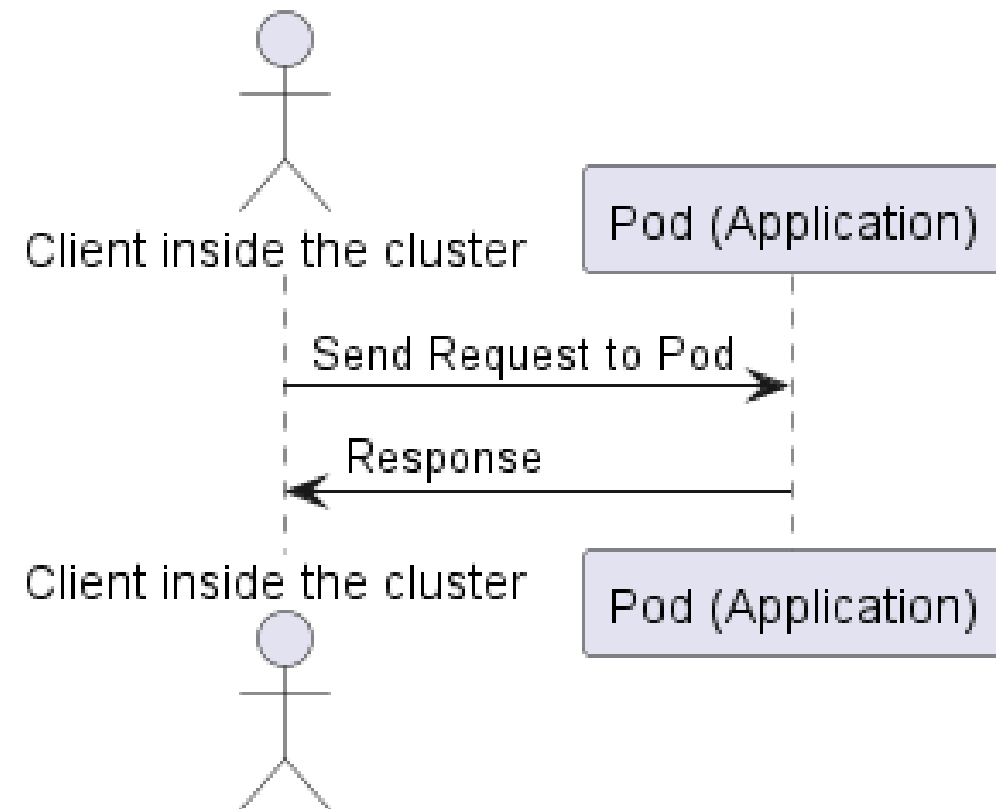
3. Konsequente Trennung von Deployment und Skalierung:

- Kubernetes trennt bewusst die Verwaltung von Deployments und die Verwaltung der Skalierung. Änderungen an der Skalierung (Anzahl der Replikate) sind separate Operationen, die nicht automatisch mit einem Rollback verbunden sind.
- Ein Rollback konzentriert sich auf das Zurücknehmen von Änderungen, die am Pod Template vorgenommen wurden, während die Skalierung bewusst unverändert bleibt.

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

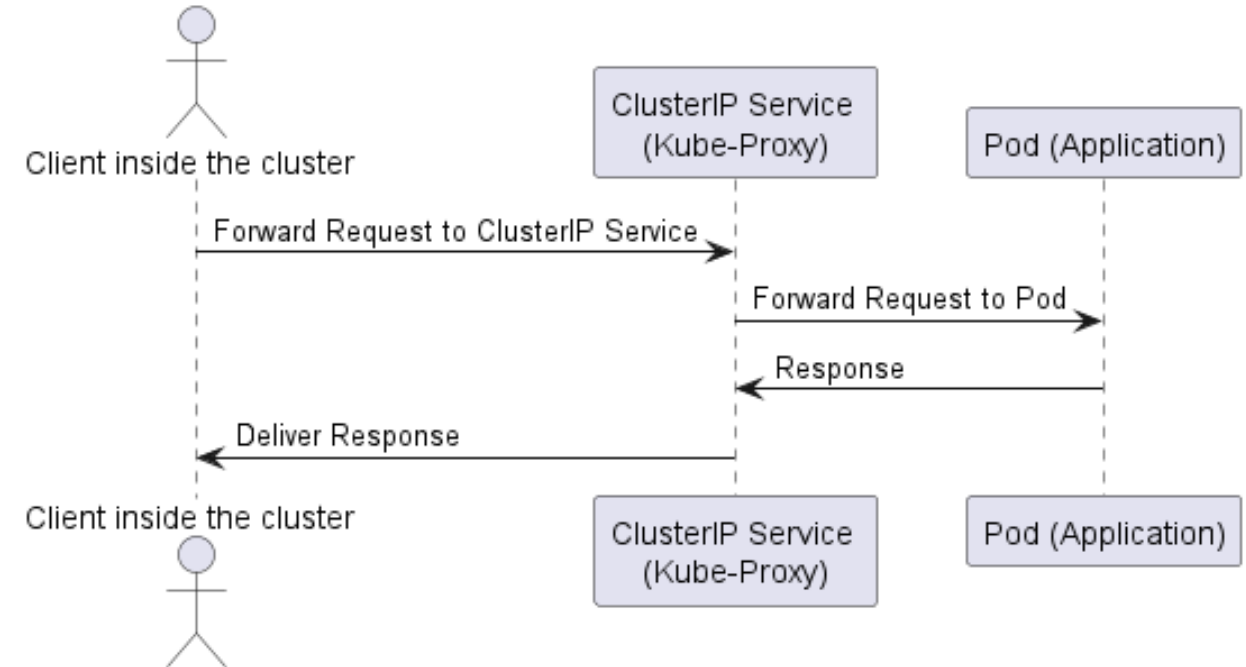
Service-Erreichbarkeit im Kubernetes-Cluster

- Pods haben ein eigenes virtuelles Netzwerk.
- Sie sind über ihre IP-Adresse erreichbar.
- Die IP-Adresse eines Pods wird in der Regel nicht festgelegt - ist daher auch nicht prognostizierbar.
- Ist die IP-Adresse bekannt, so kann ein Client im Cluster einen Pod erreichen.
- Maßnahmen sind erforderlich für die Service-Discovery.
- In manchen Kubernetes-Clustern gibt es für einen Pod auch einen DNS-Eintrag:
`<pod-name>.<namespace>.pod.cluster.local`



Abstraktionsschicht für Pods

- **Services** bilden eine Abstraktionsschicht für den Netzwerkzugriff auf eine Anwendung.
- Über den Service (kube-proxy) erfolgt ein Load-Balancing für das Anwendungs-Cluster.
- DNS-Eintrag:
`<Service-Name>. <namespace>.svc.cluster.local`



Service – Beispiel-Manifest

- ***selector***: Definiert, für welche Pods der Service verwendet wird.
- ***targetPort***: Port des Pods.
- ***port***: Port des Services.

```
$ cat 01-hashicorp-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: hashicorp-service
spec:
  selector:
    app: hashicorp-echo
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9999
```

```
~ $ curl http://hashicorp-service:80
Hallo Seminarteilnehmer
~ $ curl http://hashicorp-
service.sk.svc.cluster.local:80
Hallo Seminarteilnehmer
```

Service – Eigenschaften

- **Selector:** Definiert, für welche Pods der Service verwendet wird.
- **Type:** *ClusterIP* ist der Standard.
- **IP Family Policy:** *SingleStack* - nur IPv4
- **IP Families:** *IPv4* - nur IPv4 wird unterstützt.
- **IP:** Cluster-interne IP-Adresse.
- **IPs:** alle IP-Adressen im Cluster: nur interessant, wenn *Dual-Stack* verwendet wird.
- **Port:** Port, über den der Service erreichbar ist.
- **TargetPort:** Port, über den der Pod erreichbar ist.
- **Endpoints:** IP-Adresse und Ports der Pods.
- **Session Affinity:** *None* - zustandslose Kommunikation.
- **Internal Traffic Policy:** *Cluster* - Service nimmt clusterweit Anfragen entgegen - nicht nur pro Node.

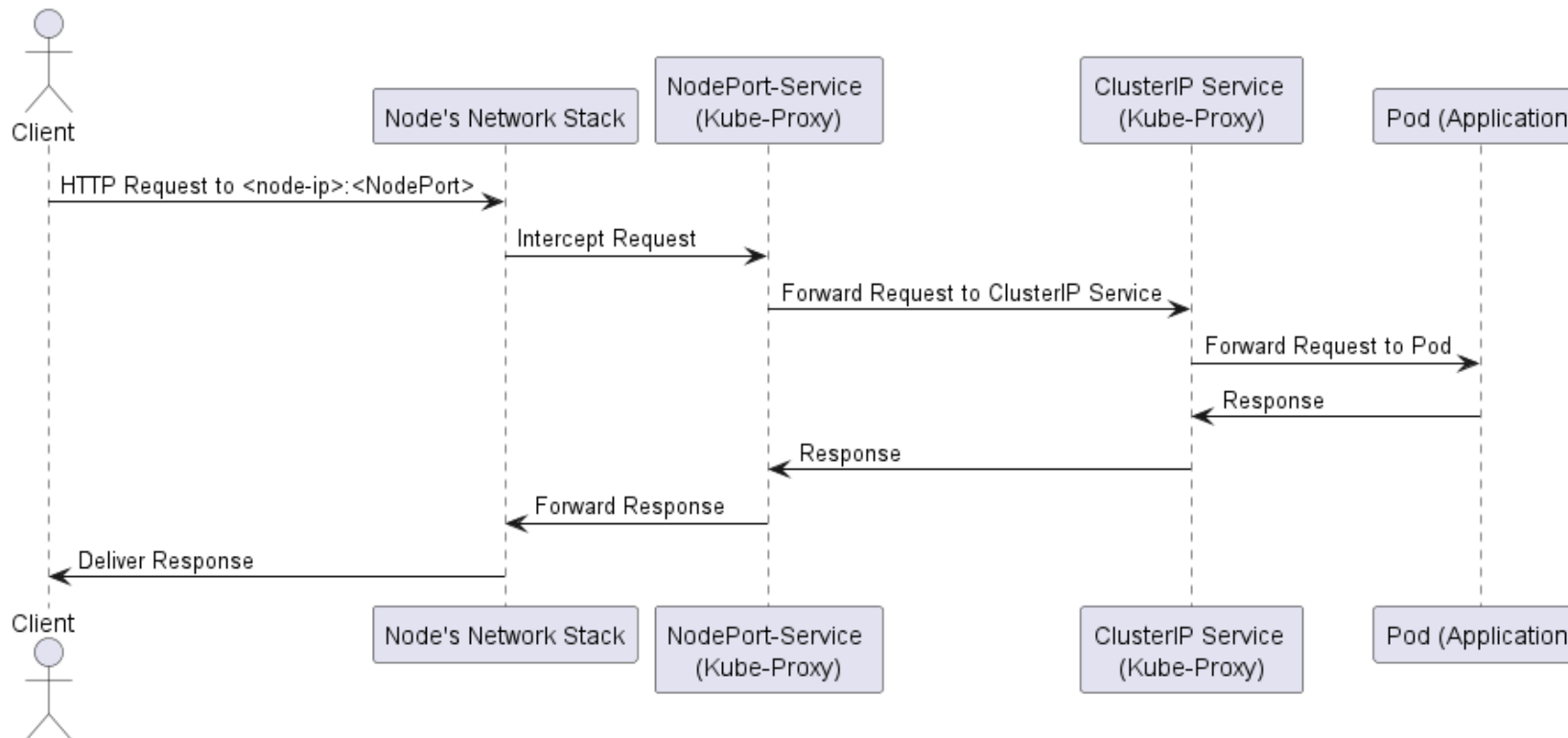
```
$ kubectl describe service hashicorp-service
Name: hashicorp-service
Namespace: sk
Labels: <none>
Annotations:
cloud.google.com/neg: {"ingress":true}
Selector: app=hashicorp-echo
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 34.118.232.96
IPs: 34.118.232.96
Port: <unset> 80/TCP
TargetPort: 9999/TCP
Endpoints: 10.44.0.15:9999
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>
```

Cluster-IP und Ports

- **Service erhält per Default eine IP-Adresse** (ClusterIP).
- IP-Adresse ist nur innerhalb des Clusters routingfähig.
- **Anfragen** an diese IP-Adresse werden **an einen der Pods weitergeleitet**.
 - automatisches Load-Balancing (random) wird durchgeführt.
- **Ports** für Anfragen und Zielport **können angegeben werden**.
- Per Default sind Port und Zielport identisch.

Service Typen – type: NodePort

- **Zusätzlich zur ClusterIP** wird auf **jedem Host ein statischer Port für den Service** geöffnet
- Anfragen an <NodeIP>:<NodePort> werden an den *Service* (ClusterIP) weitergeleitet
- Über diese Adresse ist **der Service von außerhalb des Clusters erreichbar**
- Standardmäßig wird ein zufälliger Port aus dem Bereich 30000 – 32767 ausgewählt



Service – mit Node-Port

- ***nodePort***: Port des Services.
Der Port auf dem Node muss verfügbar sein!
- Ohne Angabe von nodePort wählt der Service einen freien Port aus.

```
apiVersion: v1
kind: Service
metadata:
  name: hashicorp-service-nodeport
spec:
  type: NodePort
  selector:
    app: hashicorp-echo
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9999
      # optional nodePort: <Node-Port>
```

Service – mit Node-Port - Eigenschaften

- **NodePort:** Externer Port, unter dem der Service erreichbar ist.
- IP-Adressen der Nodes ermitteln:
\$ kubectl get node -o wide
- Extern mit curl ausprobieren:
\$ curl http://172.20.20.28:31973
Hallo Seminarteilnehmer

```
$ kubectl describe service hashicorp-  
service-nodeport  
Name:                                hashicorp-service-  
nodeport  
Namespace:                            sk  
Labels:                                <none>  
Annotations:  
cloud.google.com/neg: {"ingress":true}  
Selector:                            app=hashicorp-echo  
Type:                                NodePort  
IP Family Policy:                    SingleStack  
IP Families:                          IPv4  
IP:                                    34.118.226.54  
IPs:                                    34.118.226.54  
Port:                                  <unset> 80/TCP  
TargetPort:                           9999/TCP  
NodePort:                          <unset> 31973/TCP  
Endpoints:  
10.44.0.18:9999,10.44.1.20:9999  
...
```

Service Typen – LoadBalancer

- Macht **Service über den Load-Balancer des Cloud-Providers erreichbar.**
- Bietet Unterstützung für die wichtigsten Provider: AWS, Azure, GCP, usw.
- **ClusterIP und NodePort werden automatisch angelegt.**
- Im Load-Balancer werden die NodeIPs und der Port hinterlegt.
- Es wird der Load-Balancer des Cloud-Providers genutzt.
- DNS-Eintrag des Load Balancer wird im Objekt Service hinterlegt.
- Außerhalb von Kubernetes sind weitere Schritte erforderlich, um einen gewünschten DNS-Namen für den Service zu definieren.

Service – Loadbalancer – Beispiel

- Durch Angabe des Typs werden getriggert:
 - eine Cluster-IP
 - ein Node-Ports
 - eine externe IP-Adresse
 - ein Load-Balancer-Eintrag
- **Ein externer DNS-Name wird nicht automatisch erzeugt.**

```
apiVersion: v1
kind: Service
metadata:
  name: hashicorp-service-loadbalancer
spec:
  type: LoadBalancer
  selector:
    app: hashicorp-echo
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9999
```

Service – Load-Balancer – Eigenschaften

- Provisionierung mit neuer IP-Adresse dauert etwas.
- IP-Adressen der Nodes ermitteln:
\$ kubectl get service hashicorp-service-loadbalancer
- Extern mit curl ausprobieren:
\$ curl http://35.242.215.44:80
Hallo Seminarteilnehmer

```
Events:
  Type          Reason              Age   From
  Message
  ----
  -----
  Normal        EnsuringLoadBalancer  59m
service-controller Ensuring load balancer
  Normal        EnsuredLoadBalancer  59m
service-controller Ensured load balancer
```

```
$ kubectl describe service hashicorp-
service-loadbalancer
Name:                hashicorp-service-
loadbalancer
Namespace:           sk
Labels:              <none>
Annotations:
cloud.google.com/neg: {"ingress":true}
Selector:            app=hashicorp-echo
Type:                LoadBalancer
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  34.118.228.4
IPs:                 34.118.228.4
LoadBalancer Ingress: 35.242.215.44
(VIP)
Port:                <unset> 80/TCP
TargetPort:          9999/TCP
NodePort:            <unset> 31634/TCP
Endpoints:
10.44.0.18:9999,10.44.1.20:9999
Session Affinity:    None
```

Service-Typen - Zusammengefasst

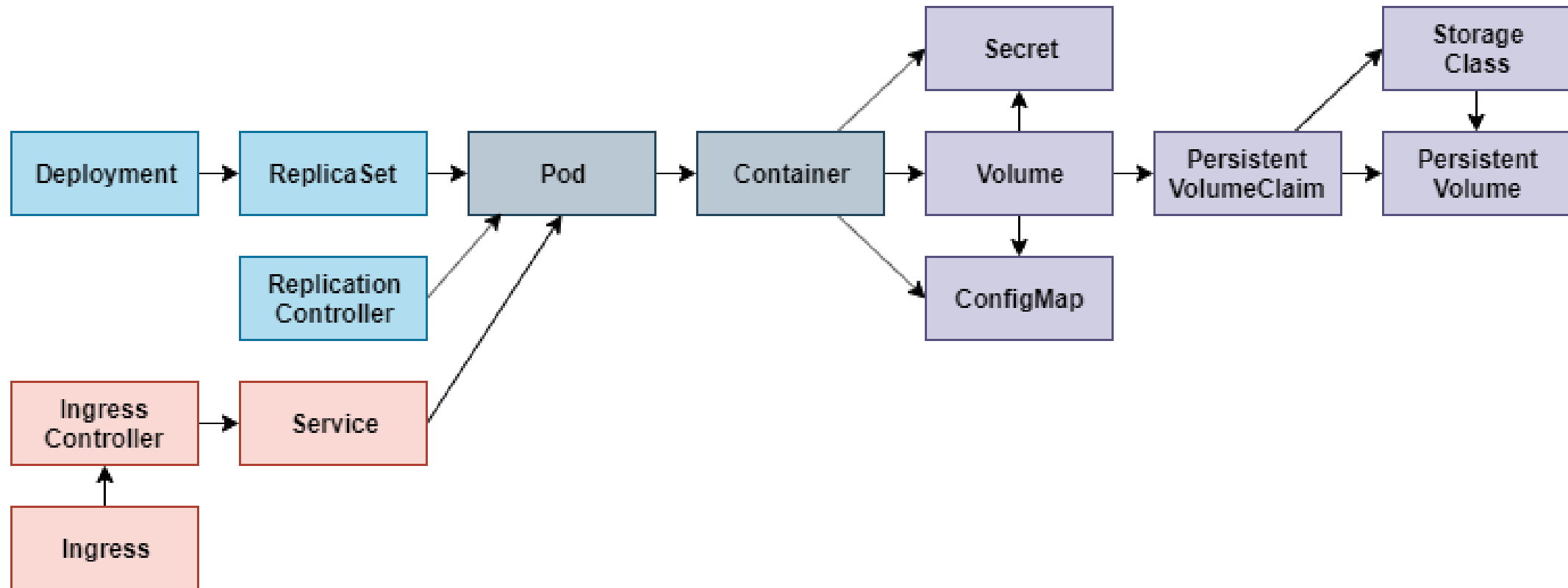
Feature	ClusterIP	NodePort	LoadBalancer
Zugriff	Intern (im Cluster)	Extern (über Nodes)	Extern (über Load Balancer)
Zielport	Interne IP	NodePort (30000-32767)	Load Balancer IP/DNS
Anwendungsfall	Interne Dienste	Externer Zugriff ohne Load Balancer	Externer Zugriff mit Load Balancer
Verfügbarkeit	Nur Cluster intern	Jeder Node-Port	Load Balancer vom Cloud-Provider verwaltet

Service API - Imperative Erzeugung eines Services

- Deployment erzeugen:
\$ kubectl create deployment *my-deployment* --image=IMAGE:TAG --port=8080
- Anzahl Replicas auf 2 skalieren:
\$ kubectl scale deployment *my-deployment* --replica 2
- Service-Objekt (Typ: ClusterIP) direkt auf der Basis eines Deployments per Commandline erzeugen:
\$ kubectl expose deployment *my-deployment*
- Auflisten der Service-Objekte im Cluster:
\$ kubectl get service [--namespace *my-namespace*] -o wide

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

- Cluster-extern werden Services über *Ingress* veröffentlicht.
- Routing werden in Ingress-Manifesten konfiguriert.
- Features:
 - **Extern-erreichbare URL für Services**
 - **Load-Balancing**
 - **SSL/TLS-Terminierung**
 - **Named-based Virtual Hosting**
Ingress kann anhand von Domännennamen zu unterschiedlichen Back-End-Services des Clusters routen.
- Services mit anderen Protokollen als HTTP und HTTPS werden über die Service-Typen *NodePort* oder *LoadBalancer* veröffentlicht.



- Der **Ingress Controller** ist das Backend für *Ingress*.
- Ingress-Controller werden in einem Kubernetes-Cluster z. B. als Deployment installiert.
- Auswahl der zur Verfügung stehenden *IngressController*:

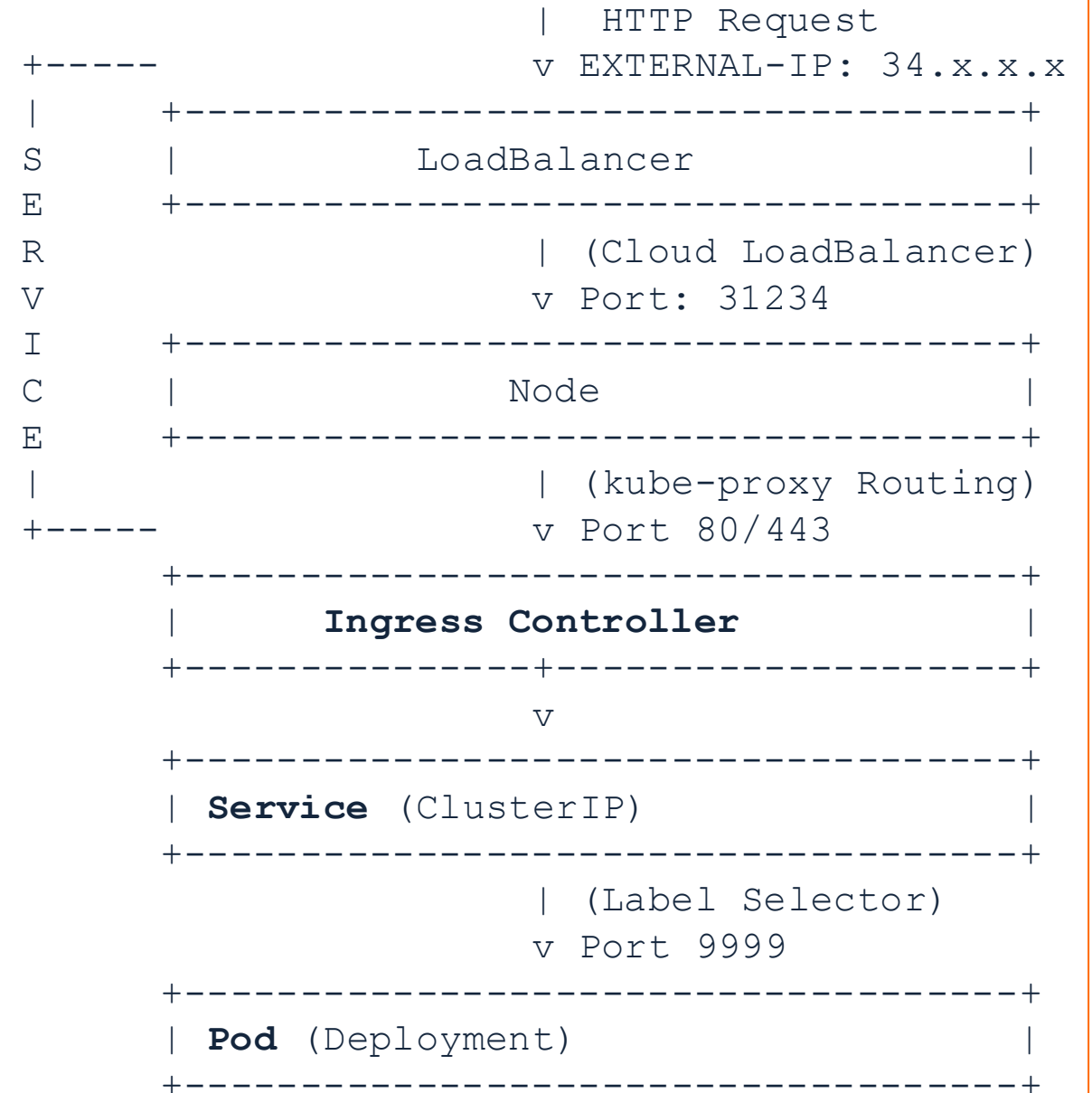
Vom Kubernetes Projekt unterstützte Controller	Extern entwickelte Controller
GCE L7 Load Balancer Controller	F5 BIG-IP Controller
AWS ALB Ingress Controller	HAProxy Ingress Controller
NGiNX Ingress Controller	Traefik

- Die verschiedenen Controller verhalten sich nicht alle nach der Referenzspezifikation und damit evtl. leicht unterschiedlich.
- In einem Cluster können mehrere *IngressController* deployed werden, der zu verwendende wird im Manifest *Ingress* ausgewählt.

Kommunikation über Ingress

- Der Weg eines HTTP-Requests über Ingress.
- Der Ingress-Controller selbst verwendet einen Service vom Typ Loadbalancer.
- Dadurch bekommt der Ingress-Controller Requests über eine externe IP-Adresse.
- Es kann mehrere Ingress-Controller geben. Auswahl über IngressClasses:

\$ kubectl get ingressclasses.networking.k8s.io



Ingress - Manifest

- Voraussetzung ist ein Service *hashicorp-service*.
- ***ingressClassName***: Zuordnung zum Ingress-Controller.
- ***pathTyp***: Prefix - Anhand des Host-Namens erfolgt die Zuordnung Request - Service.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hashicorp-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: user-sk-hashicorp.kubernetes-
      cluster-183.schulung.gcp.ordix.de
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: hashicorp-service
                port:
                  number: 80
```

Ingress - Beschreibung

- Ingress ist erst dann vollständig eingerichtet, wenn die Pods mit den Ports angezeigt werden, die den Request bekommen.

```
$ kubectl describe ingress hashicorp-ingress
Name:                hashicorp-ingress
Labels:              <none>
Address:             35.246.181.0
Ingress Class:       nginx
Default backend:     <default>
Rules:
  Host
  Path  Backends
  ----
  ----
  user-sk-hashicorp.kubernetes-cluster-
183.schulung.gcp.ordix.de

/    hashicorp-service:80
(10.44.0.18:9999,10.44.1.20:9999)
...
```

Ingress - Ingress-Class

- Es kann mehrere Ingress-Controller in einem Kubernetes-Cluster geben.
- Die Zuordnung eines Ingress zu einem Ingress-Controller erfolgt über das Attribut *ingressClassName*.
- Detaillierte Prozess-Anweisungen für den Ingress-Controller werden über Annotationen gesteuert.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hashicorp-ingress
spec:
  ingressClassName: traefik
  ...
```

```
$ kubectl get ingressclasses.networking.k8s.io
NAME                                CONTROLLER
PARAMETERS    AGE
gitlab-nginx   k8s.io/ingress-nginx   <none>
15d
nginx          k8s.io/ingress-extern   <none>
16d
```

Ingress - Konfiguration durch Annotationen

- Beispiel für die Konfiguration mit Annotation: URL-Rewriting: Die von Client angegeben URL wird nicht 1:1 an den Service weitergegeben.
- Beispiel: Der Context-Pfad 'rest' wird vorangestellt.

```
apiVersion: v1
kind: Pod
metadata:
  name: echo-server
  labels:
    app: echo-server
spec:
  containers:
    - name: echo
      image: ealen/echo-server
      ports:
        - containerPort: 80
```

```
apiVersion: v1
kind: Service
metadata:
  name: echo-service
spec:
  selector:
    app: echo-server
  ports:
    - port: 80
      targetPort: 80
```

Ingress - Konfiguration durch Annotationen mit nginx

- Die Annotation *rewrite-target* definiert, wie die Target-URL umgeschrieben wird.
- Der Positions-Parameter *\$1* bezieht sich auf den regulären Ausdruck in *path*.
- Vorsicht: Annotationen sind abhängig von der Implementierung des Ingress-Controllers.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: echo-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target:
/rest/$1
spec:
  ingressClassName: nginx
  rules:
    - host: user-sk-rest.kubernetes-cluster-
      183.schulung.gcp.ordix.de
      http:
        paths:
          - path: /(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: echo-service
                port:
                  number: 80
```

Ingress - Konfiguration durch Annotationen mit traefik

- Mit *traefik* als Ingress-Controller wird für das Forwarding des Path eine Komponente vom Typ *Middleware* benötigt!
- Diese Komponente wird über eine Annotation im Ingress-Manifest referenziert.

```
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: echo-rewrite
spec:
  replacePathRegex:
    regex: ^/(.*)
    replacement: /rest/$1
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: echo-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.middlewares:
user-sk-echo-rewrite@kubernetescrd
spec:
  ingressClassName: traefik
  rules:
    - host: user-sk-rest.kubernetes-cluster-
180.schulung.gcp.ordix.de
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: echo-service
              port:
                number: 80
```

- „Veröffentlicht“ einen einzelnen Service, wenn Ingress keine passende Routing-Regel findet.
- Alle Anfragen werden unverändert an den Service weitergeleitet.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx
spec:
  ingressClassName: nginx
  defaultBackend:
    service:
      name: nginx
      port:
        number: 80
```


Ingress – Strategie: Simple Fanout (1/2)

- Ein **Simple Fanout Ingress** ist ein Ingress-Typ in Kubernetes, der den eingehenden Datenverkehr basierend auf dem URL-Pfad an verschiedene Backends weiterleitet. Dies ermöglicht es, verschiedene Pfade auf unterschiedliche Services im Kubernetes-Cluster zu verteilen.
(Mehrere Services über eine IP bereitstellen)
- **Verteilung erfolgt auf Basis der URI (Path)**
- *pathType* gibt an, wie der Pfad matchen soll:
 - *Exact*: Path muss exakt der URI entsprechen.
 - *Prefix*: Path muss der URI oder ihrem Anfang entsprechen.
 - *ImplementationSpecific*: Matchen des Path wird der Implementierung überlassen.

Ingress – Strategie: Simple Fanout (2/2)

```
metadata:
  ...
spec:
  ingressClassName: nginx
  rules:
    - host: user01.schulung.gcp.ordix.de
      http:
        paths:
          - path: /nginx
            pathType: Prefix
            backend:
              service:
                name: nginx
                port:
                  number: 80
          - path: /apache
            ...
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-fanout-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
    - host: example.com
      http:
        paths:
          - path: /foo
            pathType: Prefix
            backend:
              service:
                name: foo-service
                port:
                  number: 80
          - path: /bar
            pathType: Prefix
            backend:
              service:
                name: bar-service
                port:
                  number: 80
```

Ingress – Strategie: Name-based Virtual Hosting

- Mehrere Services über eine IP bereitstellen
- Weiterleitung basiert auf dem **HTTP-Host-Header**

```
...
spec:
  ingressClassName: nginx
  rules:
    - host: nginx.user01.schulung.gcp.ordix.de
      http:
        paths:
          - path: "/"
            pathType: Prefix
            backend:
              service:
                name: nginx
                port:
                  number: 80
    - host: apache.user01.schulung.gcp.ordix.de
      ...
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: name-based-virtual-hosting-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: foo.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: foo-service
                port:
                  number: 80
    - host: bar.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: bar-service
                port:
                  number: 80
```

- Ingress kann mittels TLS abgesichert werden.
- Dazu muss ein TLS-Zertifikat inkl. Key in einem Secret hinterlegt werden.
- Dieses wird im Ingress-Objekt referenziert.
- Ingress vom Typ TLS überwacht den Port 443.
- **Das Zertifikats-Management ist normalerweise automatisiert (über Cert-Manager)!**

- Manuelles Zertifikats-Management:

```
apiVersion: v1
kind: Secret
metadata:
  name: nginx-tls
data:
  tls.crt: <base64 encoded cert>
  tls.key: <base64 encoded key>
type: kubernetes.io/tls
```

- Ingress Manifest:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: webserver
spec:
  ingressClassName: nginx
  tls:
    - hosts:
      - nginx.user01.schulung.gcp.ordix.de
      secretName: nginx-tls
  rules:
    - host: nginx.user01.schulung.gcp.ordix.de
      ...
```

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

Horizontales Skalieren von Pods

- Leistungsanforderungen von Applikationen sind zeitabhängig.
- Horizontale Skalierung: **Anpassung der Anzahl von Pods.**
- Auf die geänderten Anforderungen kann man manuell reagieren, z. B.:

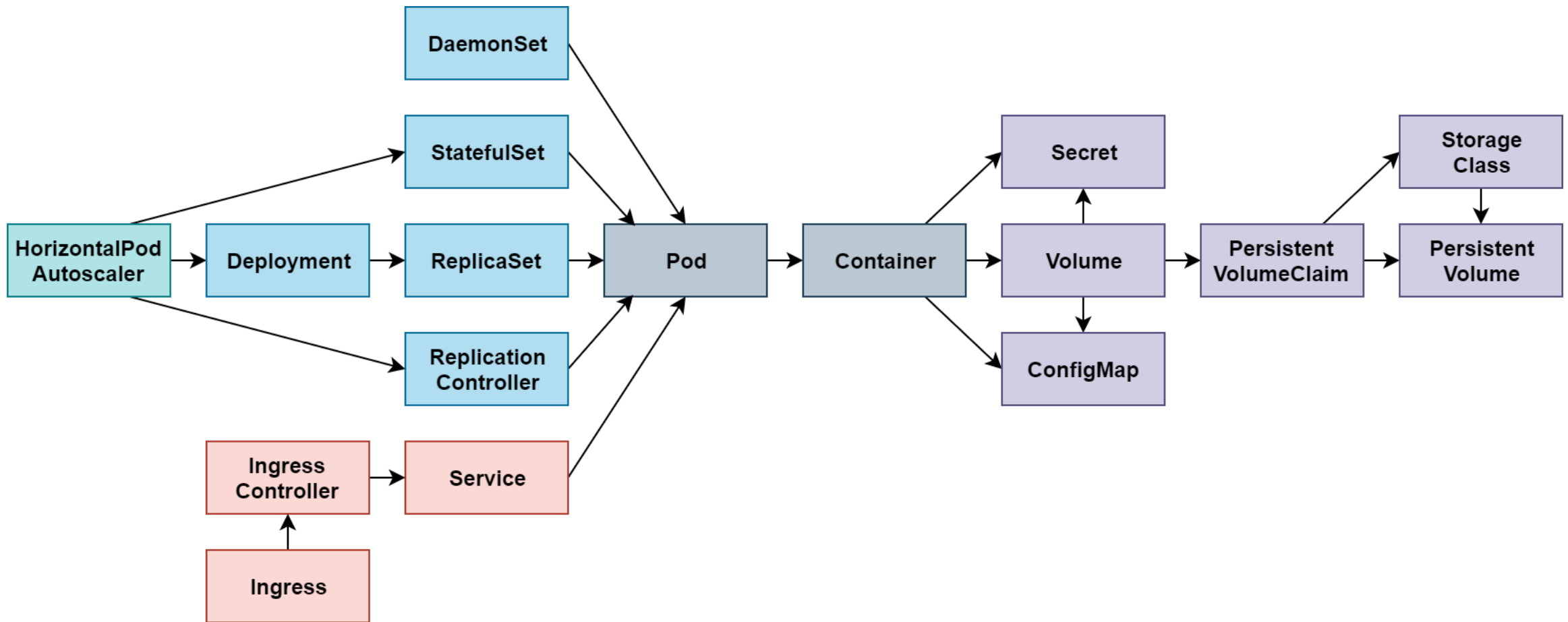
```
# kubectl scale deploy web --replicas=5
```

```
# kubectl apply -f <angepasste Konfiguration>
```
- Kubernetes kann auch **automatisch** auf geänderte Anforderungen reagieren!
- Dafür wird der *HorizontalPodAutoscaler* verwendet.

Der **HorizontalPodAutoscaler (HPA)** ist eine Kubernetes-Ressource, für die automatische Skalierung der Anzahl von Pod-Replikas. HPA reagiert auf CPU-Auslastung oder andere benutzerdefinierte Metriken.

Hauptfunktionen des HorizontalPodAutoscaler

1. **Automatische Skalierung:** HPA überwacht kontinuierlich die Auslastung der Pods und passt die Anzahl der Replikate an, um die Leistung zu optimieren und die Anforderungen zu erfüllen.
2. **CPU-Auslastung und benutzerdefinierte Metriken:** Standardmäßig skaliert HPA basierend auf der CPU-Auslastung. Es kann jedoch auch so konfiguriert werden, dass es auf benutzerdefinierte Metriken wie Speicherverbrauch oder andere Anwendungsmetriken reagiert.
3. **Effizienz und Kostenmanagement:** Durch das automatische Hinzufügen und Entfernen von Pods entsprechend der Auslastung hilft HPA dabei, die Ressourcennutzung zu optimieren und Kosten zu reduzieren.



- Automatische Skalierung der Pods von:
 - ReplicationControllern / ReplicaSets
 - Deployments
 - StatefulSets
- Version 1: Skalierung auf Basis der CPU-Nutzung.
- Version 2: Skalierung auf Basis weiterer Metriken, z. B. Memory-Auslastung oder applikationsspezifischen Metriken.
- Controller Manager überprüft in regelmäßigen Abständen die Ressourcenmetriken der überwachten Pods.
- Metrics-Server muss aktiviert und für den Pod Ressourcenanforderungen (*.spec.containers.resources.requests*) definiert sein.

- Erstellen eines HPA für ein Deployment:

```
# kubectl autoscale deploy nginx --min=1 --max=5 --cpu-percent=70
```

```
# kubectl get horizontalpodautoscalers | hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
nginx	Deployment/nginx	0%/70%	1	5	1	5m

- Bedeutung:

- Es steht mindestens ein Pod zur Verfügung.
- Maximal werden 5 Pods erstellt.
- Durchschnittliche Ziel-CPU-Auslastung.
 - Up- bzw. Down-Scaling, wenn die Ziel-Auslastung über bzw. unterschritten wird.

- Beispiel:

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: nginx
spec:
  minReplicas: 1
  maxReplicas: 5
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx
  targetCPUUtilizationPercentage: 70
```

- Anzeige der HPA-Konfiguration in Version v2:

```
# kubectl get hpa.v2.autoscaling nginx -o yaml
```

HPA - Version 2

- V2 bringt syntaktische Erweiterungen:
- *stabilizationWindowSeconds*: Vermeidet hektische Veränderung.
- *scaleUp*: Wähle die kleinste Empfehlung des HPA.
- *scaleDown*: Wähle die größte Empfehlung des HPA.

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
      - type: Percent
        value: 100
        periodSeconds: 60

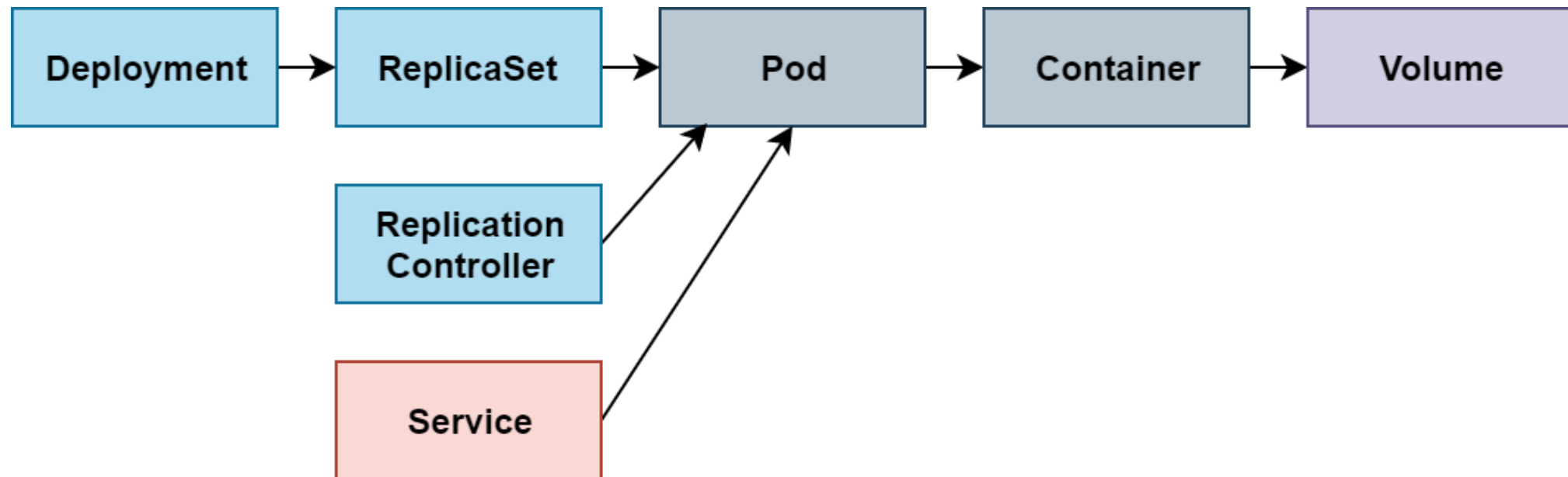
  scaleDown:
    stabilizationWindowSeconds: 10
    policies:
      - type: Percent
        value: 50
        periodSeconds: 60
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: nginx
spec:
  minReplicas: 1
  maxReplicas: 5
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx

metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

← (Fortsetzung)

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)



- Container und die Daten innerhalb eines Containers sind flüchtig.
- Z. B. beim Crash eines Containers gehen alle Daten verloren.
 - `kubelet` startet einen neuen Container mit einem sauberen Environment.
- Für die **Persistierung von Daten** werden **Volumes** genutzt.
- Ein *Volume* ist im Wesentlichen ein Verzeichnis auf einem Storage-Medium.
- *Volumes* werden an einen Pod attached und von den Containern geteilt.
- Damit können Daten über Container-Neustarts hinweg konserviert werden.
- Jeder Container im Pod muss unabhängig definieren, wo das *Volume* eingebunden werden soll.

- Kubernetes unterstützt **verschiedenste *Volume* Typen**.
- Nicht direkt unterstützte Storage-Anbieter können über das Container Storage Interface (CSI) eingebunden werden.
- Ein Pod kann gleichzeitig verschiedene *Volume* Typen nutzen.
- **Volume-Typ legt fest, welches Storage-Medium bzw. -Backend** genutzt wird.
- Auswahl verschiedener Volume Typen
 - Storage der diversen Cloud-Provider
 - glusterfs
 - iscsi
 - nfs
 - secret

- ***emptyDir***
 - Ein leeres Volume wird erstellt, sobald der *Pod* auf einem Node gescheduled wird.
 - Teilt den Lebenszyklus mit dem Pod → wird der Pod gelöscht werden auch die Inhalte des Volumes gelöscht.
 - Nutzt das Filesystem des Nodes oder tmpfs als Backend.
- ***hostPath***
 - Teilen einer Datei oder eines Verzeichnisses vom Host mit dem Pod.
 - Eng mit dem Host verknüpft, wenn der Pod gelöscht wird, bleiben die Daten auf dem Host erhalten.
 - Wenn ein Pod auf einem anderen Worker-Node verschoben wird, bekommt er dessen Daten gemountet.
 - Wird für Infrastruktur-Aufgaben verwendet.
- ***local***
 - Repräsentiert ein lokales Verzeichnis auf dem Workernode.
 - Wird **als *Persistent-Volume*** definiert (s.u.)
 - Das Persistent-Volume wird so konfiguriert, dass die Pods an den Worker-Node gebunden werden.

Beispiel emptyDir

- *application* schreibt Daten.
- *logstash* liest Daten.
- Gemeinsames Volume *emptyDir*.

```
apiVersion: v1
kind: Pod
metadata:
  name: app-logstash-pod
spec:
  containers:
    - name: application
      image: busybox
      command: ["sh", "-c", "while true; do echo $(date)
'Application log message' >> /shared-logs/app.log; sleep 5; done"]
      volumeMounts:
        - name: shared-data
          mountPath: /shared-logs
    - name: logstash
      image: busybox
      command: ["sh", "-c", "tail -f /shared-logs/app.log"]
      volumeMounts:
        - name: shared-data
          mountPath: /shared-logs
  volumes:
    - name: shared-data
      emptyDir: {}
```

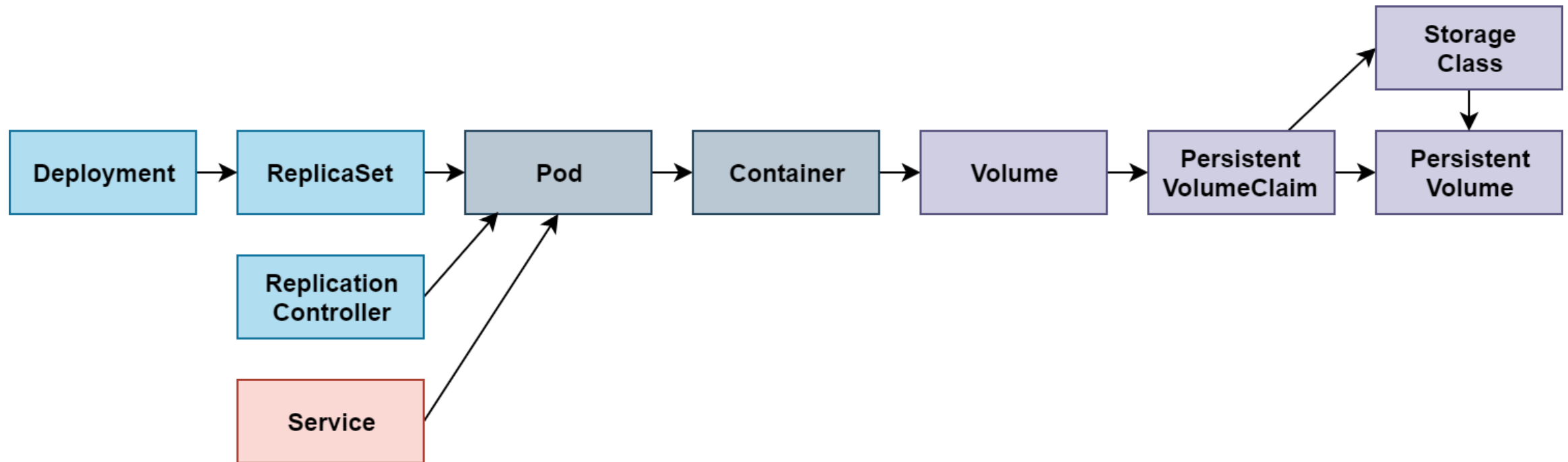
Beispiel hostPath

- *hostPath* greift auf das Dateisystem des Worker-Nodes zu.
- Wegen der Bindung an den Worker-Node nur sehr bedingt einsetzbar.

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-example
spec:
  containers:
    - name: busybox
      image: busybox
      command: ["sh", "-c", "while true; do cat /mnt/host-
data/file.txt; sleep 5; done"]
      volumeMounts:
        - name: host-volume
          mountPath: /mnt/host-data
  volumes:
    - name: host-volume
      hostPath:
        path: /data/on-host
        type: Directory
```

- **Abstraktion zwischen zur Verfügung Stellen und Einbinden von Storage.**
- Es werden 2 API-Ressourcen genutzt.
- ***PersistentVolume***
 - **Storage, der vom Administrator im Cluster zur Verfügung gestellt wird.**
 - Storage kann statisch oder dynamisch bereitgestellt werden.
 - Der Storage-Lifecycle ist unabhängig von dem der Pods.
 - Es können verschiedene Backends (local, NFS, AWS EBS Volume, ...) genutzt werden.
- ***PersistentVolumeClaim***
 - **Anfrage des Users zur Nutzung von persistentem Speicher.**
 - Greift auf PersistentVolumes zu.
 - Gewünschte Größe und der Modus (read-write/read-only) werden mitgegeben.

Persistente Volumes – Zusammenhang



- *StorageClasses* dienen der **Beschreibung unterschiedlicher Storage-Profile**.
- Können z. B. nach Geschwindigkeit, Redundanz usw. aufgeteilt sein.
- Bietet Flexibilität, wenn externer Storage automatisch bereitgestellt, verwendet und wieder freigegeben werden soll.
- *ReclaimPolicy*:
 - Delete: Wird der Volume-Claim gelöscht, werden auch die Daten gelöscht.
 - Retain: Beim Löschen des Volumen-Claims bleiben die Daten erhalten. Manuelles Aufräumen erforderlich.

```
$ kubectl get storageclasses.storage.k8s.io
```

NAME	PROVISIONER	RECLAIMPOLICY	...
dynamic-rwo	pd.csi.storage.gke.io	Delete	...
premium-rwo	pd.csi.storage.gke.io	Delete	...
standard	kubernetes.io/gce-pd	Delete	...
standard-rwo (default)	pd.csi.storage.gke.io	Delete	...

Beispiel Default Volume-Claim

- Ohne Angabe von Persistent-Volume-Claim wird der Standard eingesetzt.
- *accessModes*:
- *ReadWriteOnce*: Nur ein Node kann dieses Volume zum Schreiben mounten.
- *ReadOnlyMany*: Mehrere Nodes können dieses Volume zum Lesen mounten.
- *ReadWriteMany*: Mehrere Nodes können dieses Volume zum Lesen und Schreiben mounten.
Wird normalerweise nicht unterstützt.
- Status ist Pending, bis das Volume verwendet wird.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-nginx-webdata
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

```
$ kubectl describe persistentvolumeclaim pvc-nginx-webdata
Name:                pvc-nginx-webdata
Namespace:           user-sk
StorageClass:        standard-rwo
Status:              Pending
```


Beispiel Volume-Claim nutzen

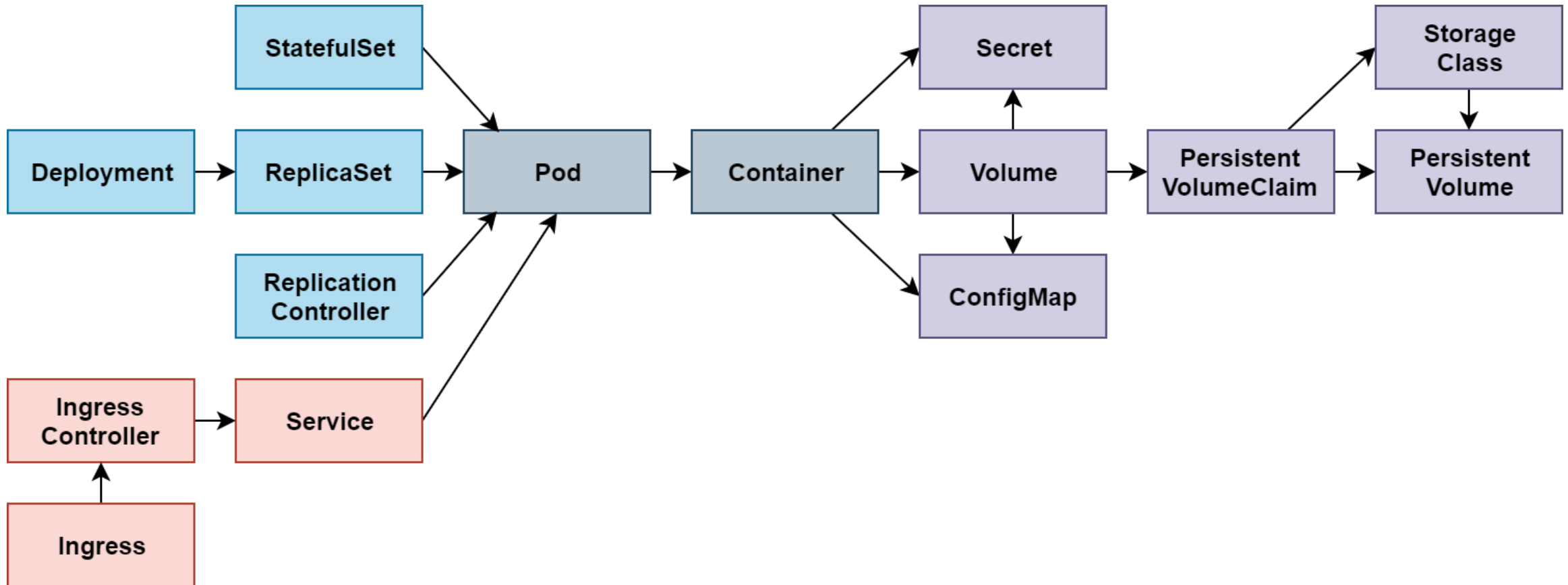
- Ohne Angabe von Persistent-Volume-Claim wird der Standard eingesetzt.
- Status des Persistent-Volumes (s.u.).

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
      volumeMounts:
        - mountPath: /usr/share/nginx/html
          name: pv-www
  volumes:
    - name: pv-www
      persistentVolumeClaim:
        claimName: pvc-nginx-webdata
```

NAME	CAPACITY	ACCESS MODE	RECLAIM POLICY	STATUS	CLAIM	STORAGECLASS
pvc-3cc6ec15-34ca-491f-90e6-1b2d593a852a	1Gi	RWO	Delete	Bound	user-sk/pvc-nginx-webdata	standard-rwo

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

StatefulSets – Zusammenhang



- *StatefulSets* dienen der Bereitstellung von zustandsbehafteten Applikationen.
- Diese haben besondere Anforderungen in den folgenden Bereichen:
 - **Identität:** Eindeutiger und vorhersagbarer Name für den Pod.
 - **Reihenfolge:** Fixe Position des Pods im Verbund mit garantierter Startreihenfolge.
 - **Netzwerk:** Vorhersehbare und stabile Netzwerk-Identität.
 - **Storage:** Dedizierte und persistente Volumes pro Pod.

StatefulSets – Identität und Reihenfolge

- Pods, die durch ein *StatefulSet* erstellt werden, haben einen **eindeutigen und vorhersagbaren Namen** (Name des *StatefulSets* + Index).
- Auf Basis des Index werden die Pods nacheinander gestartet bzw. terminiert.
- Eindeutiger Name der Pods ist Grundlage für die Bereitstellung des Netzwerks und Storage.

```
# kubectl get po
```

NAME	READY	STATUS	RESTARTS...
web-0	1/1	Running	0
web-1	0/1	ContainerCreating	0

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
            name: web
```

- **Erzeugung der Netzwerk-Identitäten** erfolgt über einen ***headless Service***
- Schema für DNS-Einträge:
<pod>.<service>.<namespace>.svc.cluster.local
- **Service wird vor dem *StatefulSet* angelegt** und muss im StatefulSet als *.spec.serviceName* referenziert werden.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  clusterIP: None
  selector:
    app: nginx
```

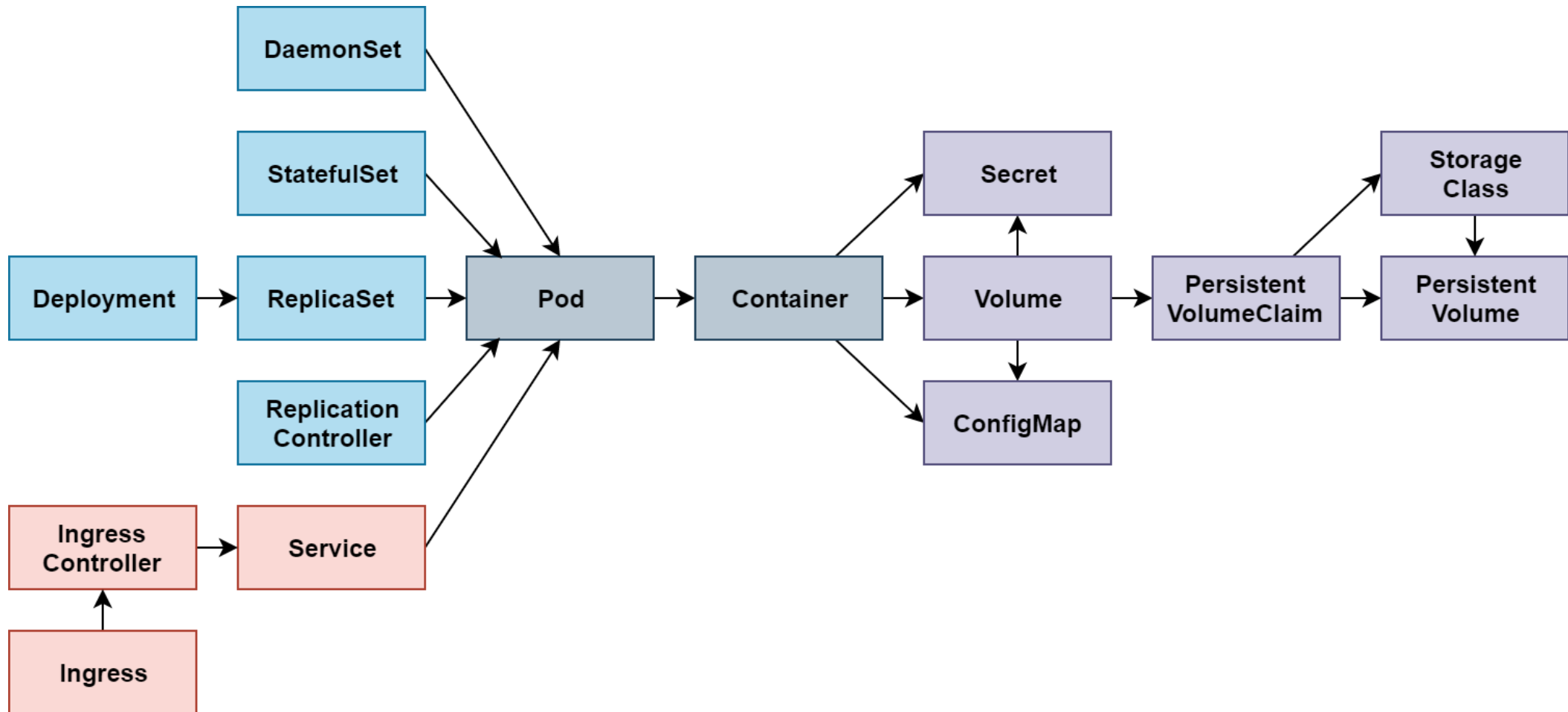
StatefulSets – Storage

- ReplicaSets stellen allen Pods dasselbe persistente Volume zur Verfügung.
- Bei *StatefulSets* erhält **jeder Pod ein dediziertes PV über eine PVC**.
- Dafür wird das Element *.spec.volumeClaimTemplates* verwendet.
- PVCs (und PVs) werden beim Down-Scaling vom *StatefulSet* nicht gelöscht.

```
template:
  ...
volumeClaimTemplates:
  - metadata:
      name: www
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 1Mi
```

- Labels
- Anwendung horizontal skalieren (ReplicaSets)
- Anwendungs-Deployment (Deployment)
- Anwendung über das Netzwerk verfügbar machen (Service, Ingress)
- Load-Balancing und Hochverfügbarkeit (Horizontal-Pod-Autoscaler)
- Konfiguration von persistentem Speicher (Grundlagen zu persistentem Speicher, Verwendung)
- Datenbanken im Kubernetes-Cluster installieren (StatefulSet)
- Cluster-Funktionalität erweitern (Erläuterung zum Einsatz von DaemonSets)

DaemonSets – Zusammenhang



- Ermöglichen, dass **auf definierten Nodes eine Kopie eines Pods** läuft.
- Werden damit in der Regel vom Admin **für Infrastruktur-Services** genutzt.
- Pod wird **auf jedem Node gestartet, außer `.spec.template.spec.nodeSelector` ist definiert.**
- Auf neuen Nodes im Cluster werden Pods automatisch gestartet.
- Start erfolgt nicht über den Scheduler, sondern dem *DaemonSet-Controller*, da für die Pods der nodeName bereits beim Erstellen gesetzt wird.
 - Pods können auf den Nodes laufen, bevor der Scheduler gestartet wurde.
- Typische Beispiele für die Nutzung:
 - Cluster-Storage-Daemon, z. B. glusterd.
 - Log-Collection-Daemon, z. B. fluentd oder Filebeat.
 - Node-Monitoring-Daemon, z. B. Prometheus Node Exporter.

DaemonSets – Erreichbarkeit

- **Service**

- Einen Service mit dem gleichen Selektor wie der DaemonSet erstellen.
 - Erreichbarkeit eines zufälligen Pods über den Service.

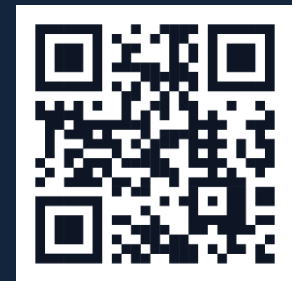
- **DNS**

- Einen headless Service mit dem gleichen Selektor wie der DaemonSet erstellen.
 - Pods sind über separate A-Records erreichbar.

- **hostPort**

- Spezifizierung eines hostPorts für die Pods.
- *spec.template.spec.containers.ports.hostPort*.
- Pods sind über die IP-Adresse des Nodes und den angegebenen Port erreichbar.

**Vielen Dank für
Ihre Aufmerksamkeit**



ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00
Fax: 0180 1 67349 0

seminare@ordix.de
<https://www.seminare.ordix.de/>